Active Learning for a Fictional Assay

Biology / Bayesian Optimization | Implementation and interview guide

Purpose and intended demonstration

Select informative points on a purely mathematical assay response surface using a Gaussian-process surrogate. This is a computational biology learning project with synthetic or fictional inputs, not a clinical or experimentally validated system.

How the implementation works

Use a fixed RBF kernel and Gaussian observation-noise term to compute posterior mean and variance. Sequentially choose the largest upper confidence bound among unobserved candidates.

Evaluation and evidence

Compare best true response found over a fixed query budget with seeded random selection. Both policies start from the same five points; hold the synthetic objective fixed.

Active Learning for a Fictional Assay

Biology / Bayesian Optimization | Implementation and interview guide

Setup and commands

Requires Python 3.10 or newer and NumPy. Create a virtual environment: python -m venv .venv. On Windows use .venv\Scripts\python.exe if activation is unavailable. Install with python -m pip install -r requirements.txt. Run python app.py, then open results/report.html in your browser. Run python -m unittest discover -v to execute tests. The default seed is 42; change it with --seed. Replace sample_input.json using --input path-to-file.json, preserving its schema. Algorithm-only projects generate synthetic arrays from the seed rather than consuming external files.

Architecture and file responsibilities

CLI arguments -> JSON input or seeded generator -> domain pipeline -> validation or evaluation -> JSON and HTML artifacts. Imported functions can be used in a notebook or a separate application. Model checkpoints preserve learned arrays where applicable; see app.py for parameter order. This package does not include a production web service. Each project is independent and includes its own helper code so the ZIP can be extracted and run alone. app.py holds the experiment and domain functions: kernel, posterior, objective, run. kit.py contains reporting, input loading and only the retrieval or generation helpers needed by this project. sample_input.json contains editable inputs. test_app.py checks domain behaviors; test_reporting.py checks metrics and HTML output boundaries. results/metrics.json records outputs and results/report.html provides a browser-readable report. Numeric projects also save arrays or learned parameters to results/artifacts.npz; inspect its keys with numpy.load. MODEL_CARD.md explains intended use and limits. .github/workflows/tests.yml runs tests and the fixture experiment. No API key is needed.

Active Learning for a Fictional Assay

Biology / Bayesian Optimization | Implementation and interview guide

Limitations and next improvements

Axes are dimensionless mathematical descriptors, not temperatures, doses, biological targets or instructions for a real experiment. Hyperparameters are fixed and no optimality guarantee is claimed. Add repeated-seed comparisons, noisy objectives, cost-aware acquisition and domain-approved real data with a separate prospective validation plan.

Interview walkthrough

Present the problem and demonstrate one normal example, then one failure or boundary case. Explain the baseline or fixture mode, the split or validation design, and one limitation before discussing an extension. Be ready to trace data through the functions listed above. Explain why the chosen metric matches the task and why the available results cannot establish production performance. Add your own documented experiment and be precise about what you implemented or changed. These are AI-assisted educational projects; filenames and synthetic scores are not evidence of client work.

Research context and reading

These topics were selected from research directions reviewed on 22 October 2026. Recent activity suggests useful skills to develop; it is not a forecast of hiring, commercial success or scientific validity. The implementation is a small educational baseline and does not reproduce or claim the performance of any cited foundation model. Reading: Benchmarking algorithms for generalizable single-cell perturbation response prediction (2025/2026): https://www.nature.com/articles/s41592-025-02980-0.

Verified build results

project

"Active Learning for a Fictional Assay"

active best

1.0954237809160943

random best

1.086947496111454

grid oracle best

1.0954237809160943

queries

23

initial queries

5

scope

"Dimensionless fictional response surface; no wet-lab parameters"

See results/metrics.json for all observations and TEST_RESULTS.txt for the executed tests. All datasets and replay responses shipped here are synthetic. Results can vary across seeds and numerical library versions.